

面向对象程序设计 C++ 实验6报告

一、选择题

第1题

题目：关于常量成员函数和常量对象的说法，哪些正确？

答案：B、D

分析：

- 常量对象的数据成员在其生命周期内不能通过普通方式修改。
- 常量成员函数中的 `this` 可视为指向常量对象，不能修改普通数据成员。
- 常量成员函数也不能直接调用同一对象的非常量成员函数。

第2题

题目：如果用户定义了自己的拷贝构造函数，哪些成员函数/运算符编译器将不会自动合成？

答案：A、B

分析：

- 用户自定义拷贝构造函数后，移动构造函数不会再被隐式生成。
- 由于类中已经声明了构造函数，编译器也不会再自动生成默认构造函数。

第3题

题目：关于面向对象编程和类的继承，下列说法正确的是哪一项？

答案：C

分析：

- `public` 继承下，基类的 `private` 成员不会成为派生类可直接访问的成员。
- 派生类成员函数可以访问基类的 `protected` 成员，所以 A 错。
- 构造函数不能被直接继承成普通成员函数，析构函数也不能继承，因此题目中最严谨的正确项是 C。

第4题

题目：判断引用和函数重载示例中的正确选项。

答案：B、C、D

分析：

- 形参 `x` 虽然类型为右值引用，但在函数体中它本身是左值，所以 `int& y = x;` 合法。
- `const int&` 可以绑定右值，因此 B 正确。
- 非 `const` 左值引用不能绑定字面量 `2`，因此 C 正确。
- `z` 为 `const int&`，不能传给 `f(int&)`，也不能匹配 `f(int&&)`，因此 D 正确。

第5题

题目：判断 `dynamic_cast`、`static_cast` 与虚函数调用示例中的正确选项。

答案：B、C

分析：

- `bp` 实际指向 `Derived` 对象，调用虚函数会动态绑定到 `Derived::print()`。
- `dynamic_cast<Base*>(derived)` 是安全的向上转型。
- `static_cast<Derived*>(base)` 把实际指向 `Base` 对象的指针强制转为 `Derived*`，不安全。

第6题

题目：关于 `using`、继承方式与重载解析的说法，哪些正确？

答案：A、B、D

分析：

- 去掉 `Base0()` 后，`Derive d;` 无法默认构造，A 正确。
- 去掉 `using Base0::f;` 后，`Base0::f(double)` 不再进入重载集，输出会变化，B 正确。
- 去掉 `using Base1::f;` 后，`d.f()` 无法找到无参版本，程序不能编译，所以 C 错。
- 把 `Base1` 改为 `protected` 继承后，外部不能直接访问继承来的 `g`，删去该调用后其余部分仍可编译，D 正确。

第7题

题目：判断虚函数与普通成员函数调用示例中的正确选项。

答案：A、C

分析：

- `func1()` 是虚函数，通过 `Base*` 调用时会动态绑定到 `Derived::func1()`。
- `func2()` 不是虚函数，通过 `Base*` 调用时仍绑定到 `Base::func2()`。

第8题

题目：选出关于拷贝构造函数示例中的错误选项。

答案：B、D

分析：

- 拷贝构造函数不仅在赋值时可能涉及，更常见于对象初始化、值传递和值返回。
- 该类含有动态内存，默认拷贝构造只会浅拷贝指针，可能导致重复释放，因此必须显式深拷贝。

二、编程题1

1. 设计思路

本题要求在不修改 `main.cpp` 的前提下，完成 `Shape`、`Rectangle`、`Circle` 三个类。做法如下：

- 将 `Shape` 设计为抽象基类，提供纯虚函数 `getArea()`。
- `Rectangle` 和 `Circle` 继承自 `Shape`，分别重写 `getArea()`。
- 由于输入可能是小数，矩形的宽和高使用 `double`，避免面积计算时被截断。
- 圆面积按题目要求使用 `3.14` 计算。

2. 关键代码

文件：编程题1/Shape.h

```
#ifndef SHAPE_H
#define SHAPE_H
```

```

class Shape
{
public:
    virtual double getArea() const = 0;
    virtual ~Shape() {}
};

class Rectangle : public Shape
{
private:
    double w, h;

public:
    Rectangle(double w = 0, double h = 0) : w(w), h(h) {}

    double getArea() const override
    {
        return h * w;
    }
};

class Circle : public Shape
{
private:
    double r;

public:
    Circle(double r = 0) : r(r) {}

    double getArea() const override
    {
        return 3.14 * r * r;
    }
};

#endif

```

3. 编译命令

本地实际使用命令：

```
g++ -std=c++14 -Wall -Wextra -pedantic main.cpp Shape.cpp -o main.exe
```

4. 运行结果

样例1输入：

```
4 4
1
```

本地实际输出：

```
16
3.14
```

补充验证输入：

```
2.5 4
1.5
```

本地实际输出：

```
10
7.065
```

5. 验证说明

- 当输入 `4 4` 时，矩形面积为 `4 × 4 = 16`。
- 当输入半径 `1` 时，圆面积为 `3.14 × 1 × 1 = 3.14`。
- 当输入 `2.5 4` 时，矩形面积正确输出为 `10`，说明宽高使用 `double` 后不会再发生小数截断。
- 当输入半径 `1.5` 时，圆面积输出 `7.065`，与公式计算一致。

6. 总结

本题重点是利用抽象基类和虚函数统一接口，通过 `Shape*` 实现多态调用。同时要注意数据类型选择，否则虽然能编译运行，但结果会与题意不符。

1. 设计思路

本题要求根据给定的 `main.cpp` 和 `shape.h`，补全 `Circle`、`Polygon`、`Cone`、`Cylinder` 的实现，并体现继承与多态。设计如下：

- `Shape` 作为抽象基类，定义面积接口 `area()`。
- `Shape2D` 在 `Shape` 基础上增加二维图形周长接口 `perimeter()`。
- `Shape3D` 在 `Shape` 基础上增加三维图形体积接口 `volume()`。
- `Polygon`、`Circle` 继承自 `Shape2D`。
- `Cone`、`Cylinder` 继承自 `Shape3D`。
- 将各类函数实现放回对应 `.cpp` 文件，保持头文件声明、源文件实现的常规组织方式。

2. 关键代码

文件：编程题2/shape.h

```
#ifndef SHAPE_H
#define SHAPE_H

class Shape {
public:
    double PI = 3.14159;
    virtual double area() const = 0;
    virtual ~Shape() {}
};

class Shape2D : public Shape {
public:
    virtual double perimeter() const = 0;
    virtual ~Shape2D() {}
};

class Shape3D : public Shape {
public:
    virtual double volume() const = 0;
    virtual ~Shape3D() {}
};

#endif
```

文件引用说明：

- 编程题2提交目录为 `编程题2`
- 本题实际提交文件为 `main.cpp`、`shape.h`、`circle.h`、`circle.cpp`、`polygon.h`、`polygon.cpp`、`cone.h`、`cone.cpp`、`cylinder.h`、`cylinder.cpp`
- 上述文件在报告中的完整引用路径分别为：
 - `编程题2/main.cpp`
 - `编程题2/shape.h`
 - `编程题2/circle.h`
 - `编程题2/circle.cpp`
 - `编程题2/polygon.h`
 - `编程题2/polygon.cpp`
 - `编程题2/cone.h`
 - `编程题2/cone.cpp`
 - `编程题2/cylinder.h`
 - `编程题2/cylinder.cpp`

文件： `编程题2/polygon.cpp`

```
#include "polygon.h"

#include <cmath>

Polygon::Polygon(double n, double s) : n(n), s(s) {}

double Polygon::perimeter() const
{
    return n * s;
}

double Polygon::area() const
{
    return (n * s * s) / (4.0 * std::tan(PI / n));
}
```

文件： `编程题2/circle.cpp`

```

#include "circle.h"

Circle::Circle(double r) : r(r) {}

double Circle::perimeter() const
{
    return 2.0 * PI * r;
}

double Circle::area() const
{
    return PI * r * r;
}

```

文件: `编程题2/cone.cpp`

```

#include "cone.h"

#include <cmath>

Cone::Cone(double r, double h) : r(r), h(h) {}

double Cone::volume() const
{
    return (1.0 / 3.0) * PI * r * r * h;
}

double Cone::area() const
{
    const double slantHeight = std::sqrt(r * r + h * h);
    return PI * r * r + PI * r * slantHeight;
}

```

文件: `编程题2/cylinder.cpp`


```
#include "cylinder.h"

Cylinder::Cylinder(double r, double h) : r(r), h(h) {}

double Cylinder::volume() const
{
    return PI * r * r * h;
}

double Cylinder::area() const
{
    return 2.0 * PI * r * r + 2.0 * PI * r * h;
}
```

3. 编译命令

本地实际使用命令：

```
g++ -std=c++14 -Wall -Wextra -pedantic main.cpp circle.cpp cylinder.cpp cone.cpp
polygon.cpp -o main.exe
```

4. 运行结果

样例1输入：

```
3 3 3
3 3
```

本地实际输出：

```
Polygon Perimeter: 9.0000
Polygon Area: 3.8971
Circle Perimeter: 18.8495
Circle Area: 28.2743
Cone Surface Area: 68.2602
Cone Volume: 28.2743
Cylinder Surface Area: 113.0972
Cylinder Volume: 84.8229
```

补充验证输入：

```
4 2.5 1.2
2.0 5.0
```

本地实际输出：

```
Polygon Perimeter: 10.0000
Polygon Area: 6.2500
Circle Perimeter: 7.5398
Circle Area: 4.5239
Cone Surface Area: 46.4023
Cone Volume: 20.9439
Cylinder Surface Area: 87.9645
Cylinder Volume: 62.8318
```

5. 验证说明

- 正三角形边长为 3 时，周长为 9.0000，面积为 3.8971，与公式一致。
- 半径为 3 的圆，周长与面积输出正确。
- 半径 3、高 3 的圆锥和圆柱，其表面积与体积均与公式计算结果一致。
- 增加的第二组输入进一步验证了正方形、圆、圆锥、圆柱在非整数或不同参数下的计算结果。

6. 总结

本题通过 Shape2D 和 Shape3D 区分二维、三维图形接口，再由具体类完成重写，体现了继承层次设计和运行时多态。补全空 .cpp 后，文件结构也更符合 C++ 工程习惯。

提交说明：

- 已按要求将 编程题2 中的 main.cpp、shape.h、circle.h、circle.cpp、polygon.h、polygon.cpp、cone.h、cone.cpp、cylinder.h、cylinder.cpp 打包为 实验6_编程题2提交包.zip

四、实验总结

本次实验的核心是抽象类、虚函数、继承和多态的综合使用。完成过程中不仅要保证代码能编译，还要通过本地真实运行验证结果是否符合题意。最终两道编程题均已在本地用指定命令编译通过，并通过样例和补充数据验证了计算结果的正确性，报告中的命令、代码片段和运行输出也已与当前实际代码保持一致。